Mechanical Engineering Skills: An Open Library of Agent Skills for Design, Analysis and Manufacturing

Achuth Chandrasekhar* Jesse Barkley* Peter Pak* Amir Barati Farimani†

Department of Mechanical Engineering, Carnegie Mellon University, Pittsburgh, PA 15213, USA

*Equal contribution †Corresponding author: barati@cmu.edu

Abstract

Mechanical engineers rely on dozens of specialized tools—CAD kernels, finite-element and CFD solvers, flight simulators, process maps, thermodynamic databases and research models—each with its own interface, installation and learning curve. We present *Mechanical Engineering Skills*, an open library of 32 agent skills for design, analysis and manufacturing that makes this breadth available through a single conversation, by text or, in hosts that support it, by voice. Each skill packages operating instructions, executable tools and verification procedures: the language model understands the request, selects the skill and coordinates the workflow, while deterministic tools do the computation and results are returned with the checks that support them. For the default workflows, users do not need to learn each tool's interface, train a model, or run a model on their own computer; skills load only when a request needs them and install with one command. Nineteen skills adapt published or preprint research from our laboratory, spanning text-to-CAD, linkage synthesis, CFD case setup, stress-field and lattice surrogates, equation discovery, additive-manufacturing process planning and in-process monitoring; 13 cover core engineering and teaching tasks. We describe the skill format, the design principles and a procedure for translating research into usable workflows. Five case studies, each with an explicit evidence scope and an independent rerun, show exact analytical agreement for lattice homogenization, rocket apogee within 4% of stored OpenRocket references, a reproduced equation fit that is nine times more accurate than a cubic polynomial at an unseen temperature, and a porosity-monitoring pipeline rehearsed end to end. A controlled comparison of agents with and without skills is left to future work. The library is available at https://github.com/BaratiLab/mechanical-engineering-skills.

1 Introduction

Mechanical engineering work is a chain of small, checkable steps: read a requirement, pick a model, compute, compare against a hand estimate or a known case, and decide. General-purpose language-model agents are now capable of driving such chains: they write CAD scripts, set up simulations, and read sensor data. Our own work has shown this across the life cycle of a part, from text-to-CAD [1, 2] and mechanism synthesis [3] to LLM-in-the-loop structural design [4], autonomous CFD [5], additive-manufacturing process planning [6–8] and print monitoring [9].

Two obstacles keep this research from becoming everyday practice. First, each method lives in its own code base, with its own environment, data and conventions, and is rarely packaged for someone who simply has an engineering question. Second, agents left to themselves make the kind of errors engineers cannot accept: arithmetic slips, wrong units, a correlation used outside its range, or a confident answer with no evidence behind it.

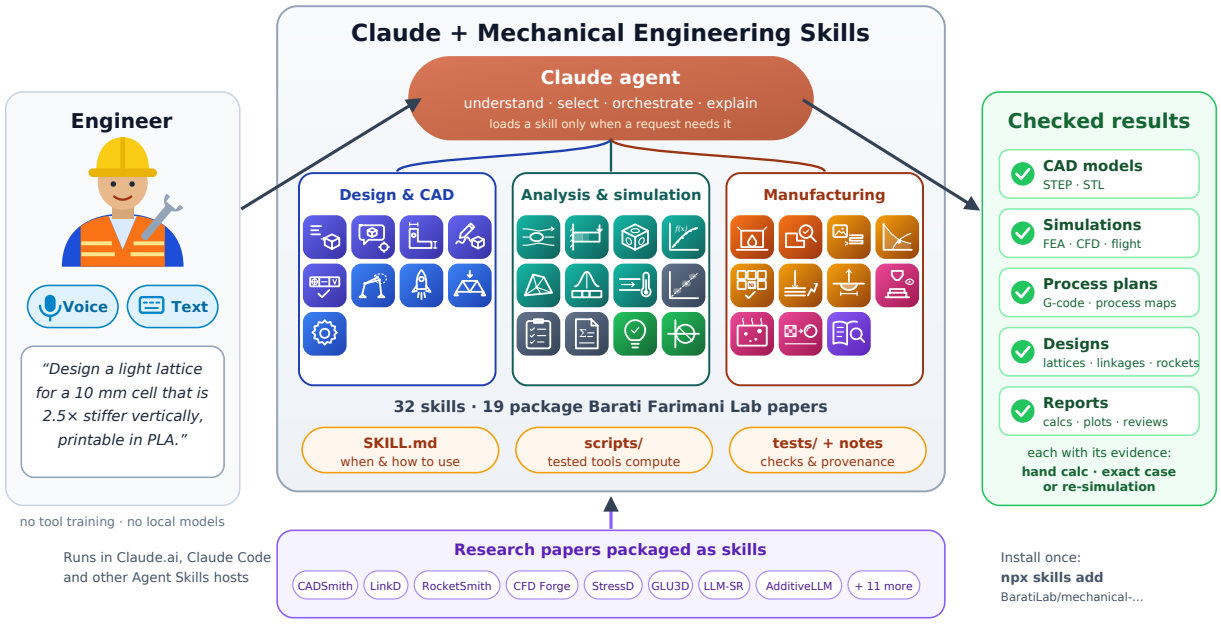


Figure 1: Mechanical Engineering Skills. An engineer states a goal by voice or text. The Claude agent understands the request, selects the relevant skills from a library of 32 (design and CAD, analysis and simulation, manufacturing), and orchestrates them. Each skill bundles instructions (`SKILL.md`), tested scripts that do the computation, and checks. The results—CAD models, simulations, process plans, designs and reports—come back with the evidence that supports them. Nineteen skills package research papers from our laboratory. No per-tool training or locally hosted model is required for the default workflows; voice input and execution depend on the host and the installed tools.

Agent Skills [10, 11] offer a lightweight remedy. A skill is a directory with a `SKILL.md` file—a short description that tells the agent when to use it, followed by instructions—plus any scripts and reference files the instructions point to. The agent sees only the one-line descriptions until a task matches, then loads the instructions, and reads deeper files or runs scripts only when needed (*progressive disclosure*). Skills are portable across Claude Code, Claude.ai and other agents that adopted the format, and they are installed with a single command. Numerical packages, external solvers and any model weights are installed separately (§4).

This paper presents *Mechanical Engineering Skills* [12], a library of 32 such skills (Fig. 1, Table 1). Our contributions are:

1. **A library** covering design, analysis and manufacturing, in which 19 skills package peer-reviewed or preprint research from our laboratory and 13 cover core engineering tasks (machine-element sizing, FEA with hand checks, tolerance stack-ups, material selection, design reviews and teaching aids).
2. **Design principles** for engineering skills: tools compute and the model orchestrates; consequential numbers are tied to explicit checks; changes to a source method are declared; and the skill states the boundary of its validity.
3. **Access without expertise in each tool**: the whole library is reachable by text or voice in one conversation with Claude, without learning tool interfaces, training models or running models locally, illustrated by twelve end-to-end tasks (§4).

4. **Five case studies** drawn from our research—lattice inverse design, printed rockets, in-situ porosity monitoring, polymer process optimization and equation discovery—each with an explicitly stated evidence scope and confirmed by an independent rerun.

2 Motivation and prior work

Mechanical engineering is a demanding setting for language-model agents: its answers are numbers with units and tolerances, they are produced by specialized tools, and a wrong one can break a part. Three lines of work meet in this library.

Large language models acquire broad knowledge and in-context reasoning from scale [13, 14], and become agents when they interleave reasoning with actions and observations [15] or learn to call external tools [16]; surveys catalogue the resulting architectures [17]. Retrieval augments generation with documents the model was not trained on [18], and the Model Context Protocol standardizes connections to external systems [19]. Agent Skills [10, 11] add a complementary layer: packaged procedural knowledge and code that an agent loads only when relevant. In science, tool-using agents have planned and executed chemistry experiments [20, 21]; our library brings the same pattern to mechanical engineering, with an emphasis on checks.

Early studies showed that general models can help across the design-to-manufacture pipeline but make errors that require verification [22]. Subsequent work generated CAD from text with execution feedback [1, 2], coupled models with finite-element analysis for structural design [4], automated CFD setup under physics constraints [5], and applied agents to additive manufacturing, from literature question answering [23] and defect prediction [6] to alloy evaluation [7], process optimization [8], print monitoring and control [9] and complete design-to-flight workflows [24]. These systems are typically bespoke; this work packages them in a common, portable format.

Generative models, in particular denoising diffusion [25–28], now produce stress fields [29], lattice architectures [30], mechanisms [3], toolpaths [31] and super-resolved process images [32], often with U-Net backbones [33]. Machine learning also predicts melt-pool geometry [34] and porosity [35] in metal additive manufacturing [36, 37], and symbolic regression recovers interpretable laws from data [38–40]. The skills pair such models with classical references—Rosenthal's moving-source solution [41], Barrowman's stability method [42], open-source solvers [43, 44], computational thermodynamics [45], cellular-solid scaling [46, 47], numerical homogenization [48] and Bayesian optimization [49]—so that learned predictions can be checked against physics wherever the assumptions are comparable.

3 Mechanical Engineering Skills

Figure 2 shows the structure we use for every skill. The `SKILL.md` front matter carries a description written for the agent's routing decision: what the skill does, what it is based on, and the situations that should trigger it. The body is a short workflow—typically four to six numbered steps—with the commands to run and, crucially, the *judgement calls* an experienced engineer would make: which inputs to ask for, what to report alongside the headline number, and where the method stops being valid. Long material (paper notes, provenance, interpretation choices) sits in `references/` so that it costs no context unless needed.

The `scripts/` directory holds the computation. Scripts are ordinary command-line Python programs with JSON output so that the agent can parse results reliably. Examples include a periodic voxel homogenizer for lattices, a plane-stress Q4 finite-element solver, a Barrowman stability and one-dimensional flight simulator for rockets, a Rosenthal melt-pool solver, an SQLite build log with Gaussian-process suggestions for selective laser sintering, and validators for OpenFOAM cases. Each

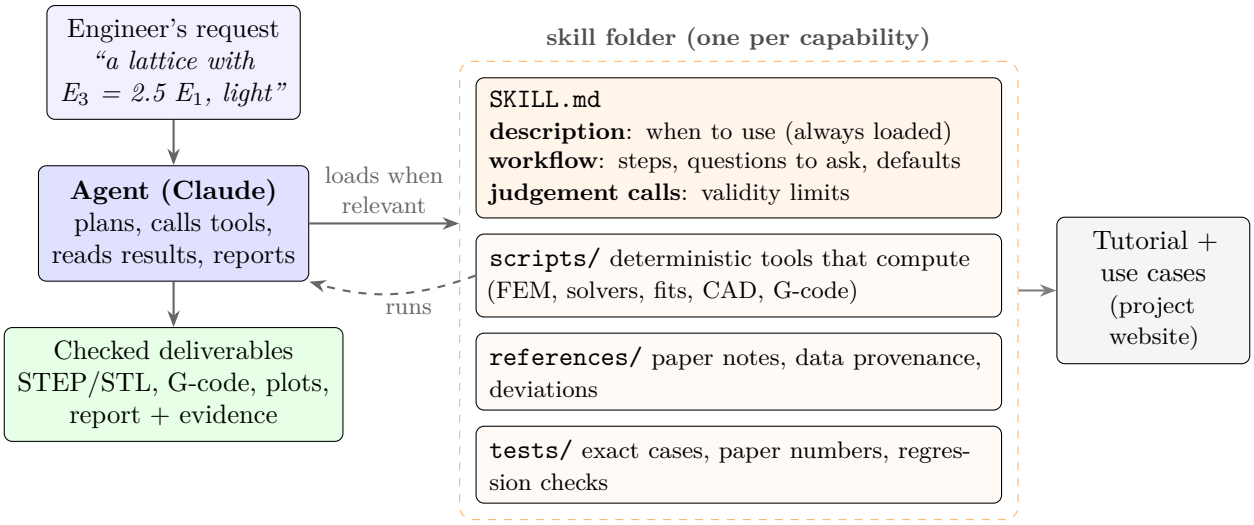


Figure 2: Anatomy of an engineering skill and how an agent uses it. Only the one-line description is always in the agent's context; instructions, scripts and references are loaded on demand. Tests and a public tutorial accompany every research skill.

research skill also ships a test suite (`tests/`) and a tutorial on the project website with executable examples and a statement of where their data come from.

Five principles shape every skill. The first is that the model orchestrates while tools compute: instructions require each consequential number to come from an executable calculation with declared inputs and units, never from the model's own arithmetic—the conclusion RocketSmith [24] reached after flying four agent-designed rockets, which we apply library-wide. The model remains responsible for what it does well: eliciting requirements, choosing a method, chaining tools and explaining results. The second is that every result travels with its check. A finite-element run is returned with a hand calculation, a lattice homogenization with the exact laminate solution, a surrogate's best candidate with a re-simulation, a CAD edit with a kernel validity and volume check. Where a published method accepted designs with a learned scorer, the skill accepts on the physics model and keeps the learned model for guidance; if a check fails or cannot run, the result is reported as unverified together with its inputs, assumptions and software versions, and agreement between tools that share assumptions is treated as consistency rather than physical validation.

Research skills follow two further rules. They keep reproduction and adaptation apart, recording which parts retain the source protocol, which use a replacement model and which have only been exercised on synthetic data, with the reason for every changed default or acceptance criterion; a working implementation is not, by itself, a reproduction of a paper's headline accuracy. And they state their boundaries where the agent will repeat them: elastic homogenization is not crash performance, a flight simulation is a design check rather than a launch approval, and a defect classifier trained on one machine family is not a process qualification. Finally, skills ask for the inputs that matter and then act. Instructions name the few inputs that change the engineering decision—printed modulus, load cases, minimum wall thickness, certification level—and give defaults for the rest, so the agent asks a small set of grouped questions instead of many generic ones and asks again only when a new ambiguity would change the result.

The library itself is summarized in Table 1, which lists all 32 skills, grouped into design and CAD, analysis and simulation, manufacturing, and teaching. Each row gives the skill's icon and name and, in plain language, what a user can ask for. Nineteen skills package research from our laboratory;

the remaining 13 cover core engineering tasks that every practitioner needs, from machine-element sizing and FEA with hand checks to tolerance stack-ups, material selection [50] and design reviews, plus two skills for teaching.

Table 1: The skill library. Every skill is installed with one command and used by asking for the result in plain language; the agent selects the skill from its description. Execution additionally requires the skill's dependencies. A citation marks the 19 skills that package a Barati Farimani Lab paper.

Skill	What you can ask for	Paper
Design and CAD		
`cadsmith-text-to-cad`	Describe a part in words; get a CadQuery model checked by exact geometry measurements and a three-view review, exported as STEP/STL.	[1]
`query2cad-freecad`	Plain-language request to a FreeCAD macro, with automatic error repair and a visual self-check.	[2]
`parametric-part-cad`	Part specification in plain English to a parametric CadQuery script, STEP/STL and a preview.	–
`sketch-to-cad`	Hand sketch, photo or video frames to a dimensioned CAD model.	–
`drawing-gdt-review`	Audit an engineering drawing for missing dimensions and GD&T errors.	–
`linkage-synthesis`	Target motion path to a four- or six-bar linkage, built joint by joint and checked kinematically.	[3]
`rocketsmith`	Hobby high-power rocket: stability, fin sizing, flight prediction, drag calibration and printable parts.	[24]
`llm-mechanical-designer`	Truss design loop: the model proposes, exact FEM evaluates, the best designs are ranked.	[4]
`machine-element-sizing`	Bolts, shafts, bearings, springs, gears and welds, with the work shown and safety factors.	–
Analysis and simulation		
`cfd-simulation-setup`	Flow problem to an OpenFOAM case, run and diagnosis; results accepted only against physics checks.	[5]
`stress-field-surrogate`	Geometry, supports and loads to a full stress field in milliseconds, with its own FEM data generator.	[29]
`lattice-inverse-design`	Target stiffness to a printable TPMS or hybrid lattice cell with its full stiffness matrix and a tiled STL.	[30]
`equation-discovery`	Test data to an interpretable equation, scored on a validation split and a final held-out test.	[40]
`quick-fea`	Linear static and modal FEA with a required hand-calculation check.	–
`tolerance-stackup`	Worst-case, RSS and Monte Carlo tolerance stack-ups with contributor charts.	–

	Skill	What you can ask for	Paper
	`thermal-fluids-calc`	Pipe pressure drop, heat exchangers and fins with real fluid properties.	–
	`material-selection`	Performance indices to a ranked material shortlist with Ashby charts.	–
	`design-review-package`	Requirements, FMEA, bill-of-materials cost roll-up and review checklist.	–
	`engineering-calc-report`	Turn calculations into a calculation report ready for sign-off.	–
Manufacturing			
	`agentscad-fdm-dfam`	Make a STEP part printable by FDM: reorientation, teardrop holes and chamfers, with verified geometry.	[51]
	`dfm-review`	Design-for-manufacture review for machining, sheet metal, molding and additive manufacturing.	–
	`image2gcode`	Photo or sketch to printer-ready G-code for 2D and 2.5D parts.	[31]
	`lpbf-defect-prediction`	Laser power and speed to keyholing, lack of fusion or balling, with process maps.	[6]
	`am-alloy-screening`	Known or new alloy to properties, a lack-of-fusion process map and a printability ranking.	[7]
	`sls-process-optimization`	Polymer SLS test results to the next batch's settings, batch by batch.	[8]
	`meltpool-geometry`	Melt-pool depth, width, length and mode from laser power, speed and alloy.	[34]
	`llm-3d-print-monitor`	Camera inspection of every layer and bounded in-print corrections on Klipper printers.	[9]
	`thermal-porosity`	Thermal images recorded during printing to pore count and location for each layer.	[35]
	`lpbf-superres`	Low-cost webcam images to high-resolution layer images and edge roughness.	[32]
	`am-literature-rag`	Page-cited answers from your own additive-manufacturing paper library.	[23]
Teaching			
	`worked-problem-generator`	Textbook-style problems with code-verified solutions.	–
	`mechanics-visuals`	Free-body diagrams, Mohr's circle, beam diagrams and S–N curves.	–

4 Engineering by voice or text with Claude

The practical value of a skill library is that it removes the cost of *reaching* a capability. Without it, using a research method or a specialized analysis means finding the code, installing its environment, learning its inputs and conventions, often downloading or training a model and providing a GPU, and then interpreting the outputs correctly. With skills, the engineer states the goal to Claude—typed in Claude.ai or Claude Code, or spoken through voice mode in the Claude apps—and Claude does the rest: it matches the request to a skill's description, loads that skill's instructions, asks for the few inputs that matter, runs the scripts and returns the result with its check.

Several properties make this practical. All 32 skills are installed with one command and are available in the same conversation, and because only each skill's one-line description stays in the model's context, on the order of a hundred tokens per skill, breadth costs almost nothing until a skill is used. Skills compose naturally, so a lattice designed with `lattice-inverse-design` can be checked for overhangs with `agentscad-fdm-dfam` and sliced without the engineer switching tools. No training on each tool is needed, because the instructions encode what an expert user of the underlying method knows—which inputs matter, which defaults are safe, which command to run and how to read its output—so the engineer never has to learn the options of a homogenizer, the input format of a flight simulator or the conventions of a thermodynamic database. The interface is spoken language as much as text: an engineer at a printer or on the shop floor can ask, hands-free, whether a parameter set risks lack of fusion or what fin span a rocket needs, and hear the answer with its supporting check. Nor is there any model to download or run locally. The language model runs in the cloud and the skills are text and lightweight scripts; where a source paper relied on a large learned model, the default path uses physics models, solvers or small fitted models that run in seconds on a CPU, such as search with exact homogenization instead of a 3D diffusion model, or a Rosenthal solution calibrated on a few tracks instead of a trained network. In Claude.ai the scripts run in Claude's own sandboxed code-execution environment, so nothing is installed on the engineer's computer; in Claude Code they run in the engineer's project. Training the larger research models remains possible, but it is never required to get an answer.

Some prerequisites remain. The installation command copies skill folders but does not install or license every underlying tool: numerical scripts need Python packages such as NumPy and SciPy, CAD workflows can need CadQuery or FreeCAD, CFD solves need OpenFOAM, thermodynamic calculations can need a licensed database, and live monitoring needs supported printer hardware. Where a tool is missing, skills fall back to validators, analytical alternatives or rehearsal data and say so. Voice reaches the full workflow where the host also gives the agent access to the installed skills and to code execution.

Table 2 follows twelve tasks from the engineer's spoken or typed request, through what Claude does with the skills, to the result it hands back, and Fig. 3 shows one exchange in full.

Table 2: Twelve engineering tasks requested of Claude by voice or text: the input, what Claude does with the skills, and the final result returned to the engineer. The table illustrates intended use; a controlled evaluation is future work (§6).

Input (voice or text)	What Claude does	Final result
VOICE *"Size the output shaft and bearings for a 2 kW gearbox at 300 rpm, steel, safety factor two."*	 `machine-element-sizing` Computes the torque (63.7 N m), sizes the shaft for combined bending and torsion fatigue, selects bearings for the required life and shows each step.	Shaft diameter and keyway, bearing designation and rated life, fatigue safety factors, and a calculation sheet.
TEXT *"Make an L-bracket for an M6 bolt pattern on 40 mm centres, 3 mm wall, printable without supports."*	 `cadsmith-text-to-cad`, `agentscad-fdm-dfam` Writes a CadQuery model, measures it with the geometry kernel and a three-view check, then reorients it and adds teardrop holes and chamfers where overhangs need them.	Validated STEP and STL files, preview images and an overhang report.
VOICE *"Will this aluminium bracket deflect more than half a millimetre under a 500 N tip load? (STEP attached)"*	 `quick-fea` Meshes the part, runs a linear static and a modal analysis, and compares the deflection with a beam hand calculation before answering.	A yes/no answer with deflection, peak stress, safety factor, first modes and the hand-check agreement.
TEXT *"Set up air flow over this heat-sink fin array at 2 m/s and tell me whether the result is converged."*	 `cfd-simulation-setup`, `thermal-fluids-calc` Builds and runs an OpenFOAM case, checks residuals, conservation and mesh sensitivity against registered criteria, and estimates the heat-transfer coefficient from correlations as a cross-check.	The case files, an accept-or-rerun decision with reasons, and pressure drop and heat transfer compared with the correlation.
VOICE *"I need a lattice infill for a 10 mm cell, about 2.5 times stiffer vertically than sideways, as light as possible, in PLA."*	 `lattice-inverse-design` Asks for the absolute stiffness and minimum wall, searches homogenized TPMS and hybrid cells, refines thickness and re-checks the best cell at higher resolution.	The cell, its full stiffness matrix and weakest-direction modulus, and a tiled watertight STL (Fig. 3).
VOICE *"I'm building a 98 mm rocket for an H motor. Will it be stable, and how high will it go?"*	 `rocketsmith` Computes the centre of pressure by Barrowman's method, sizes the fins to the target stability, simulates the flight and checks rail-exit speed, thrust-to-weight and certification level.	Stability margin, predicted apogee and peak speed, a pass/fail checklist, and printable nose cone and fin can.

Input (voice or text)	What Claude does	Final result
TEXT *"Will 280 W at 900 mm/s keyhole on Ti-6Al-4V?"*	`lpbf-defect-prediction`, `meltpool-geometry` Classifies the defect regime for the parameter set and estimates melt-pool depth and width from a calibrated moving-source model.	The predicted regime, melt-pool dimensions and a process map that marks a candidate operating window.
VOICE *"Layer 140 looks stringy on the camera—what is wrong and what should I change?"*	`llm-3d-print-monitor` Reads the layer images and the printer state, diagnoses the likely cause and proposes G-code changes within safety bounds, waiting for approval before applying them.	A diagnosis, the approved correction applied to the running print, and a log of every change.
TEXT *"Which layers of yesterday's build should go to CT first? (pyrometry and CT folders attached)"*	`thermal-porosity` Filters and aligns the thermal frames, builds per-layer temperature and dwell maps, and runs the pore-count and localization models.	Pore counts and maps for every layer, ranked for inspection, with an alignment check image.
TEXT *"Here are our last six SLS batches and test results. What should batch seven be?"*	`sls-process-optimization` Reduces the ASTM tensile and flexural data, updates the build log and proposes bounded next settings with the expected effect of each lever.	Fill energy, overlap and surface temperature for the next batch, with the rationale and a warning list from past builds.
TEXT *"Find a law for this stress–strain data that also holds at a temperature I have not tested."*	`equation-discovery` Proposes equation forms, fits their constants, ranks them on a validation split and reports the final held-out test once, for the frozen equation.	An interpretable equation with fitted constants, the physical meaning of each term and its errors.
VOICE *"Give my students three Mohr's-circle problems with worked solutions and figures."*	`worked-problem-generator`, `mechanics-visuals` Writes the problems, solves them in code to verify every number, and draws the circles and stress elements.	Problem sheet, step-by-step solutions and figures, ready to hand out.

5 Case studies

Each case states what kind of evidence it rests on: an exact analytical check, a comparison with published or stored reference values, an adaptation of the source method, or a rehearsal on synthetic data. All numbers were recomputed in an independent rerun (§5.6).

Engineer (voice): *"I need a lattice infill for a 10 mm cell that is about two and a half times stiffer vertically than sideways, as light as possible, printed in PLA."*

Agent: loads `lattice-inverse-design`; asks for the minimum printable wall and the absolute stiffness needed. The engineer confirms $E_1 = E_2 \approx 0.06\,E_s$ and $E_3 \approx 0.15\,E_s$ with printed $E_s = 3.5$ GPa; the agent searches homogenized cells, refines thickness and re-checks the best cell at higher resolution.

Agent (reply): *"A gyroid–diamond hybrid with two periods sideways gives $E_3/E_1 = 2.6$ at 26% density (about 0.2 and 0.5 GPa in PLA). Its weakest direction is still $0.057\,E_s$, so it does not go soft off-axis. The tiled $3 \times 3 \times 3$ STL is attached."*

Figure 3: An illustrative exchange based on the lattice case study (§5). The engineer names the goal; the skill supplies the procedure, the computation and the check. A stiffness ratio alone does not fix a design, so the agent asks for absolute stiffness and the printable wall; the search returns the best cell found, not a proven global minimum weight.

Table 3: Resolution sensitivity of the lattice found by search (rounded specification: H3 gyroid–diamond–double-diamond, $t = 0.744$, $h = 0.209$, $a = -0.187$, periods $(2, 2, 1)$).

Voxels	Volume fraction	E_1/E_s	E_2/E_s	E_3/E_s
24^3	0.2549	0.0551	0.0556	0.1406
32^3	0.2605	0.0578	0.0573	0.1473
48^3	0.2598	0.0573	0.0577	0.1506

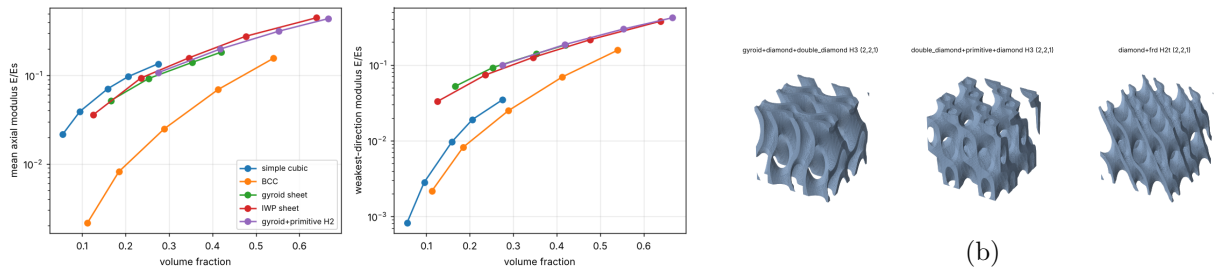
5.1 Lattice inverse design (GLU3D)

GLU3D [30] generates lattice unit cells from a target stiffness with a 3D conditional diffusion model trained on homogenized TPMS and hybrid-TPMS cells. The skill first builds the physics: implicit cells, printability filters (one periodic solid that reaches every face, no sealed cavities) and an energy-based periodic voxel homogenizer [48] that returns the full 6×6 stiffness. It uses periodic eight-node hexahedral elements, an isotropic solid with $E_s = 1$ and $\nu = 0.3$ unless specified, Voigt order (xx, yy, zz, yz, xz, xy) with engineering shear strain, and omits void voxels. Directional moduli are evaluated from the compliance tensor on 600 sphere directions, so the reported minimum is a sampled estimate.

The homogenizer reproduces a solid cell exactly and matches the exact C_{33}, C_{44} and C_{11} of two-phase laminates to a relative error below 10^{-6} across several modulus contrasts, Poisson ratios and phase fractions. A sheet gyroid at volume fraction 0.25 gives $E/E_s = 0.086$ with exact cubic symmetry. Figure 4(a) compares stiffness with simple-cubic and BCC strut lattices: at equal density the TPMS and hybrid sheets are about 5× stiffer than BCC and 3× stiffer than simple cubic in its weakest direction, though simple cubic is 1.3× stiffer along its own axes—which is why the skill reports the directional minimum.

For one-off designs the skill searches a homogenized dataset and refines sheet thickness and level-set bias. For the anisotropic target $E_1 = E_2 = 0.06$, $E_3 = 0.15$ (relative to the solid) the search and refinement took two minutes on a laptop CPU, excluding the one-time creation of a 300-cell dataset, and returned a gyroid–diamond–double-diamond hybrid with $E = (0.057, 0.058, 0.151)$ at volume fraction 0.26 (Fig. 4b). Table 3 shows the sensitivity of the rounded published specification to voxel resolution; the largest axial-modulus change from 32^3 to 48^3 is 2.2%.

The generative route adapts the paper's training recipe at reduced scale. A first run on a small dataset (300 cells, 16^3, CPU) produced unstructured fields; four changes produced structured samples: training on the periodic grid with circular padding (which makes every generated cell



(a) (b)

Figure 4: Lattice inverse design. (a) Mean axial and weakest-direction modulus against volume fraction for simple cubic, BCC, sheet gyroid, sheet I-WP and a gyroid–primitive hybrid, from the skill's homogenizer. (b) The three best cells returned by search for $E_1 = E_2 = 0.06$, $E_3 = 0.15$; anisotropy comes from two periods along x and y.

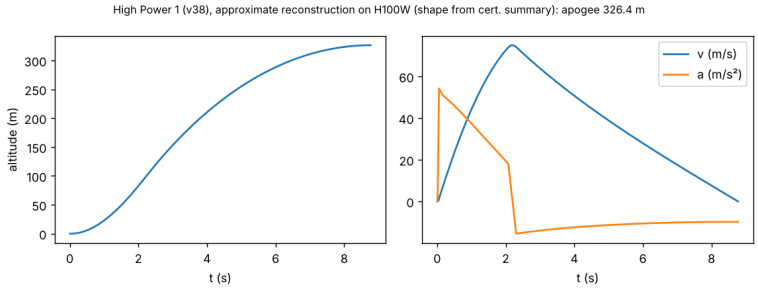


Figure 5: `rocketsmith`: simulated flight of the reconstructed HP1 rocket from the skill's Barrowman stability model and one-dimensional flight simulator; apogee differs by 3.7% from the stored OpenRocket reference.

exactly periodic), a sinusoidal-plus-MLP time embedding in place of a per-step lookup table, scaling the conditioning vector inside the encoder, and a longer learning-rate plateau patience. Stiffness error after re-homogenization remained 200–280% at this scale, against 3723 cells at 64^3 on four GPUs in the paper. The skill therefore uses search as its default design route and accepts generated cells only after re-homogenization.

5.2 Printed rockets (RocketSmith)

RocketSmith [24] designed four FDM-printed Class 2 rockets that all flew stably, with altimeter apogees at 80–84% of predictions. The skill couples a Barrowman stability calculation [42] to a one-dimensional flight simulator and ships approximate reconstructions of two of the rockets. With $C_D = 0.45$ it predicts 326.4 m and 75.0 m/s for HP1 against the stored OpenRocket [44] references of 338.8 m and 76.4 m/s, and 567.6 m and 113.7 m/s for HP4 against 570.1 m and 115.2 m/s—apogee differences of 3.7% and 0.4% (Fig. 5). Parameters the paper does not report were set or sized by the skill: HP1's fin span (162 mm) was sized to the paper's liftoff stability of 1.26 calibers, so agreement on stability is by construction, while the flight comparison is the untuned check.

Fitting an effective drag coefficient to the measured apogees of 276 m and 479 m gives $C_D = 0.84$ and 0.76, well above the 0.4–0.5 typical of smooth commercial airframes. This calibration describes those flights; surface finish, geometry differences, mass uncertainty and flight conditions are possible contributors, and the skill exposes calibration as an explicit step so that the fitted value can be carried to the next flight and tested there.

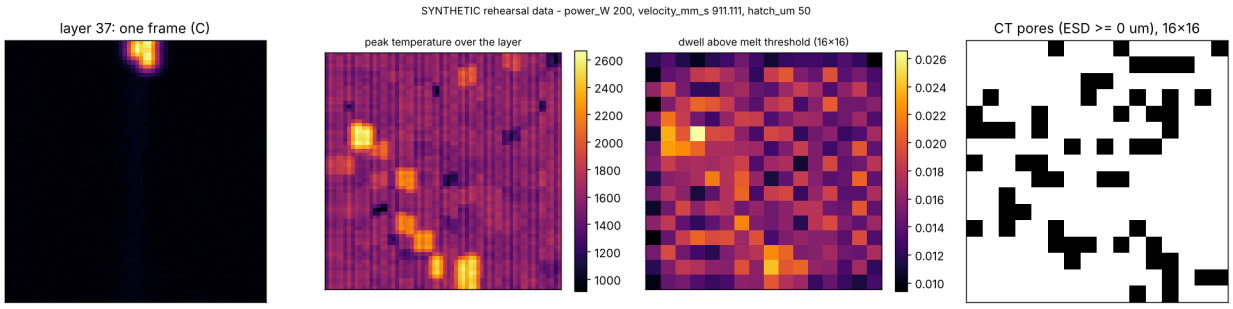


Figure 6: `thermal-porosity`: one layer of a synthetic rehearsal build. From left: a single pyrometry frame with the melt pool mid-raster; peak temperature over the layer; dwell time above the melt threshold pooled to the 16×16 label grid; and the synthetic pore labels used as ground truth.

5.3 Porosity from in-situ thermal images (ThermoPore)

Qualifying a laser powder-bed fusion part usually waits for X-ray computed tomography (CT). ThermoPore [35] asks how much of that answer is already contained in the pyrometry recorded during the build [52]. For each 30 μm layer it pairs 200 thermal frames with the CT pores in that layer and trains a CNN that counts pores ($R^2 = 0.57$, RMSE 7.84 in the source paper) and a video vision transformer [53, 54] that localizes them on a 16×16 grid (mean IoU 0.32). The skill packages the workflow around the models, which is where most practical effort goes: empty-frame filtering, resampling each layer to 200 frames, alignment of CT and thermal coordinates with an explicit check image, equivalent-diameter thresholds for which pores count, and splits within each process step as in the paper (Fig. 6). It adds interpretable tree-based baselines on per-cell peak temperature, dwell time above the melt threshold and thermal roughness, as a reference that learned models must beat.

Because the paper's pyrometry and CT data are not public, the skill ships a synthetic rehearsal build with the same structure (10 process steps, 16 layers each, 200 frames per layer; 120 training and 40 test layers). On it the baseline reaches counting $R^2 = 0.44$ (RMSE 10.1) and localization IoU 0.27. These outputs carry a synthetic-data flag and establish pipeline behaviour, not accuracy on a physical build. With a user's own build, the agent can simply be asked which layers and regions should be sent to CT first.

5.4 Sequential process optimization (SLS)

For polymer selective laser sintering [8], the skill keeps a build log of 21 published batches across three materials, reduces ASTM D638/D790 tensile and flexural tests, and suggests the next batch with a bounded Gaussian-process expected-improvement step [49]. This numerical suggester is an adaptation of the source agent workflow: it excludes results flagged as non-reproducible, keeps the surface temperature at least 3 °C below the melt peak and limits how far one batch can move from the best so far.

To test it without hindsight we replayed the PA12 GF campaign, removing all later batches before each of 11 decisions (A–C through A–M) and asking for three ranked suggestions with strength as the objective. None of the 33 suggestions matched the next batch on all three settings (fill energy, overlap and surface temperature). Agreement was directional: after A–C every suggestion moved to 50% overlap, as batch D did, and after A–E the suggestions included 28 mJ/mm, the fill energy later used in batch I. Because suggested batches that were never printed have no outcome, the replay cannot show that following the suggester would have performed better; its role is to propose a bounded next step for the engineer to judge by the printed result.

Table 4: Fixed-skeleton replay on the LLM-SR stress–strain benchmark (normalized MSE). The scores reproduce the numerical fit of the discovered skeleton, not the original search campaign.

Model	Train	In-domain	Held-out temperature
Discovered eight-parameter skeleton	0.02096	0.02464	0.07362
Cubic polynomial	0.16141	0.14952	0.67952
Linear regression	0.38018	0.37661	1.31432

5.5 Equation discovery (LLM-SR)

LLM-SR [40] lets a model propose equation skeletons as code while an optimizer fits the constants. The skill evaluates proposals in a time-limited, isolated Python subprocess after static source checks (a guard against mistakes, not a security sandbox) and fits constants by BFGS. On the LLM-SR stress–strain benchmark, the eight-parameter skeleton

$$\hat{\sigma}(\epsilon, T) = (p_0 + p_1 T + p_6 T^2)\,[1 - \exp\{-p_2(1 + p_7 T)\epsilon\}] - (p_3 + p_4 T)|\epsilon|^{p_5} \tag{1}$$

reaches normalized MSE 0.021, 0.025 and 0.074 on the training, in-domain and held-out-temperature splits (2161, 1442 and 738 rows), against 0.161, 0.150 and 0.680 for a cubic polynomial and 0.380, 0.377 and 1.31 for linear regression (Table 4); the original repository's evaluator gives the same training error. Every term has a physical reading: an elastic rise, a saturation stress that falls with temperature, and necking softening. The held-out temperature (0.667) lies inside the training range (0.067–1.0), so the result shows generalization to an unseen condition by interpolation, not extrapolation.

The replay also exposed a methodological flaw that we corrected in the skill: its prompt builder had shown the held-out score to the model while it proposed equations, so that score could not serve as an untouched final test. The skill now uses the in-domain split for feedback and selection, hides the held-out split from every search command, and reveals it once, through an explicit `final` step, for the equation the user has frozen; tests enforce this behaviour.

5.6 Independent rerun and evidence scope

All case-study numbers were recomputed in a separate rerun from library commit `46cfac2` and LLM-SR commit `41c2123` (macOS 15.7.7, arm64; Python 3.9.6, NumPy 2.0.2, SciPy 1.13.1, scikit-learn 1.6.1, PyAMG 5.3.0), by direct script execution without modifying the implementations. The five test suites passed all 68 assertions that can run without optional dependencies (15 rocket, 17 lattice, 12 thermal-porosity, 12 SLS and 12 equation-discovery); CadQuery export and PyTorch checks were skipped rather than counted. Table 5 summarizes what each case establishes and where its evidence stops. The rerun exercises the tools; it does not measure skill routing, natural-language reliability or voice operation, which are the subject of the controlled evaluation proposed below.

6 Limitations and future work

The case studies verify selected executable components and document how each research method was adapted; they do not yet show library-wide agent reliability or an improvement over an agent with the same tools but no skills, and they do not reproduce all 19 source papers. Synthetic rehearsals test pipeline behaviour and cannot support accuracy claims on physical builds, and learned components rehearsed at small scale remain unvalidated at the source papers' scale.

Table 5: Evidence scope of the five case studies.

Workflow	What the evidence establishes	Where it stops
Lattice	Exact solid and laminate agreement; resolution sensitivity; search result re-homogenized	No full-scale diffusion training or physical coupon tests
RocketSmith	Apogee and velocity within 4% of stored OpenRocket references	Reconstructed geometry; OpenRocket not rerun; no new flight
ThermoPore	Complete preprocessing, labelling and baseline pipeline on synthetic data	No source pyrometry/CT data; deep-model accuracy not reproduced
SLS	ASTM data reduction; 11 chronological replays with directional agreement	Retrospective; no outcomes for unprinted suggestions
LLM-SR	Discovered skeleton and baselines refitted on the original benchmark files	Original model-driven search not rerun; held-out split is interpolation

The next step is a controlled evaluation. Task definitions and acceptance criteria will be frozen before execution; the same agent with the same tool access will be run with and without skills under equal budgets and repeated across seeds; and design, analysis and manufacturing tasks will be scored on constraint satisfaction, numerical correctness, failed or skipped checks, unsupported claims, completion time and cost, including spoken requests. An ablation that removes the verification instructions will measure their contribution.

Several skills depend on external software (OpenFOAM, FreeCAD, Thermo-Calc), licensed data or hardware (a Klipper printer, a powder-bed machine); they ship validators, mock devices or rehearsal data, but real use requires the tools. Instructions make assumptions and checks visible, but whether an agent follows them, and whether the checks are adequate, must be measured. Skills inherit the limits of their methods and do not replace an engineer's sign-off on safety-critical decisions.

7 Conclusion

Mechanical Engineering Skills packages 32 engineering capabilities, including 19 adaptations of our research, as instructions, executable tools and verification procedures that a general agent can discover, load and use. The central idea is simple: let the model plan and explain, let tested code compute, and return every result with its check. Because the interface is natural language, this breadth is available in one conversation, by text or voice, without learning each tool, training models or running them locally. Writing the skills and rerunning their case studies also made each method's evidence explicit—and exposed and fixed a flaw in one evaluation—which suggests that a skill is a useful standard for research reproducibility as well as a delivery format. The library is open-source under the MIT license; we invite other groups to contribute skills and to help validate them.

Availability. Code, tests and tutorials:
`https://github.com/BaratiLab/mechanical-engineering-skills`;
website: `https://baratilab.github.io/mechanical-engineering-skills/`.
Install all skills with `npx skills add BaratiLab/mechanical-engineering-skills -all`.

References

[1] J. Barkley, R. Loghmani, and A. Barati Farimani. CADSmith: Multi-agent CAD generation with programmatic geometric validation. *arXiv preprint arXiv:2603.26512*, 2026.

[2] A. Badagabettu, S. S. Yarlagadda, and A. Barati Farimani. Query2CAD: Generating CAD models using natural language queries. *arXiv preprint arXiv:2406.00144*, 2024.

[3] Y. Jadhav and A. Barati Farimani. LinkD: AutoRegressive diffusion model for mechanical linkage synthesis. *arXiv preprint arXiv:2601.04054*, 2026.

[4] Y. Jadhav and A. Barati Farimani. Large language model agent as a mechanical designer. *Journal of Engineering Design*, 2026. arXiv:2404.17525.

[5] A. S. Oza, R. F. Johnson, and A. Barati Farimani. CFD Forge: A physics-constrained agentic framework for autonomous CFD. *arXiv preprint arXiv:2610.14104*, 2026.

[6] P. Pak and A. Barati Farimani. AdditiveLLM: Large language models predict defects in metals additive manufacturing. *Additive Manufacturing Letters*, 14:100292, 2025. doi: 10.1016/j.addlet.2025.100292.

[7] P. Pak, A. Chandrasekhar, and A. Barati Farimani. Agentic additive manufacturing alloy evaluation. *Additive Manufacturing Letters*, 17:100355, 2026. doi: 10.1016/j.addlet.2026.100355.

[8] P. Pak, V. Alvarado, and A. Barati Farimani. AI agentic selective laser sintering process optimization. *arXiv preprint arXiv:2608.25928*, 2026.

[9] Y. Jadhav, P. Pak, and A. Barati Farimani. LLM-3D Print: Large language models to monitor and control 3D printing. *Additive Manufacturing*, 114:105027, 2025. arXiv:2408.14307.

[10] Anthropic. Equipping agents for the real world with Agent Skills. Anthropic Engineering blog, October 2025. URL https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills.

[11] Agent Skills. Agent skills: an open format for packaging agent capabilities, 2025. URL https://agentskills.io. Published as an open standard in December 2025.

[12] Barati Farimani Lab. Mechanical engineering skills. GitHub repository, 2026. URL https://github.com/BaratiLab/mechanical-engineering-skills.

[13] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[16] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[17] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18:186345, 2024.

[18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[19] Anthropic. Introducing the Model Context Protocol, November 2024. URL https://www.anthropic.com/news/model-context-protocol.

[20] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

[21] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6:525–535, 2024.

[22] L. Makatura, M. Foshey, B. Wang, et al. How can large language models help humans in design and manufacturing? *arXiv preprint arXiv:2307.14377*, 2023.

[23] A. Chandrasekhar, J. Chan, F. Ogoke, O. Ajenifujah, and A. Barati Farimani. AMGPT: A large language model for contextual querying in additive manufacturing. *Additive Manufacturing Letters*, 11:100232, 2024. doi: 10.1016/j.addlet.2024.100232.

[24] P. Pak, J. Barkley, R. Loghmani, D. Baich, A. Pamal, and A. Barati Farimani. RocketSmith: Agentic additive manufacturing of high-powered rockets. *arXiv preprint arXiv:2606.00097*, 2026.

[25] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

[26] A. Q. Nichol and P. Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning (ICML)*, pages 8162–8171, 2021.

[27] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

[28] J. Ho and T. Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[29] Y. Jadhav, J. Berthel, C. Hu, R. Panat, J. Beuth, and A. Barati Farimani. StressD: 2D stress estimation using denoising diffusion model. *Computer Methods in Applied Mechanics and Engineering*, 416:116343, 2023. doi: 10.1016/j.cma.2023.116343.

[30] Y. Jadhav, J. Berthel, C. Hu, R. Panat, J. Beuth, and A. Barati Farimani. Generative lattice units with 3D diffusion for inverse design: GLU3D. *Advanced Functional Materials*, 34:2404165, 2024. doi: 10.1002/adfm.202404165.

[31] Z. Wang, Y. Jadhav, P. Pak, and A. Barati Farimani. Image2Gcode: Image-to-G-code generation for additive manufacturing using diffusion-transformer model. *Additive Manufacturing Letters*, 17:100374, 2026. doi: 10.1016/j.addlet.2026.100374.

[32] F. Ogoke, S. Kalambettu Suresh, J. Adamczyk, D. Bolintineanu, A. Garland, M. Heiden, and A. Barati Farimani. Deep learning based optical image super-resolution via generative diffusion models for layerwise in-situ LPBF monitoring. *Additive Manufacturing*, 107:104790, 2025. doi: 10.1016/j.addma.2025.104790.

[33] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

[34] P. Akbari, F. Ogoke, N.-Y. Kao, K. Meidani, C.-Y. Yeh, W. Lee, and A. Barati Farimani. MeltpoolNet: Melt pool characteristic prediction in metal additive manufacturing using machine learning. *Additive Manufacturing*, 55:102817, 2022. doi: 10.1016/j.addma.2022.102817.

[35] P. Pak, F. Ogoke, A. Polonsky, A. Garland, D. S. Bolintineanu, D. R. Moser, M. Arnhart, J. Madison, T. Ivanoff, J. Mitchell, B. Jared, B. Salzbrenner, M. J. Heiden, and A. Barati Farimani. ThermoPore: Predicting part porosity based on thermal images using deep learning. *Additive Manufacturing*, 95:104503, 2024. doi: 10.1016/j.addma.2024.104503.

[36] T. DebRoy, H. L. Wei, J. S. Zuback, T. Mukherjee, J. W. Elmer, J. O. Milewski, A. M. Beese, A. Wilson-Heid, A. De, and W. Zhang. Additive manufacturing of metallic components – process, structure and properties. *Progress in Materials Science*, 92:112–224, 2018.

[37] W. E. King, H. D. Barth, V. M. Castillo, G. F. Gallegos, J. W. Gibbs, D. E. Hahn, C. Kamath, and A. M. Rubenchik. Observation of keyhole-mode laser melting in laser powder-bed fusion additive manufacturing. *Journal of Materials Processing Technology*, 214:2915–2925, 2014.

[38] M. Schmidt and H. Lipson. Distilling free-form natural laws from experimental data. *Science*, 324:81–85, 2009.

[39] S.-M. Udrescu and M. Tegmark. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6:eaay2631, 2020.

[40] P. Shojaee, K. Meidani, S. Gupta, A. Barati Farimani, and C. K. Reddy. LLM-SR: Scientific equation discovery via programming with large language models. In *International Conference on Learning Representations (ICLR)*, 2025. Oral; arXiv:2404.18400.

[41] D. Rosenthal. The theory of moving sources of heat and its application to metal treatments. *Transactions of the American Society of Mechanical Engineers*, 68:849–866, 1946.

[42] J. S. Barrowman. The practical calculation of the aerodynamic characteristics of slender finned vehicles. M.s. thesis, The Catholic University of America, 1967.

[43] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, 12:620–631, 1998.

[44] S. Niskanen. Development of an open source model rocket simulation software. Master's thesis, Helsinki University of Technology, 2009.

[45] J.-O. Andersson, T. Helander, L. Höglund, P. Shi, and B. Sundman. Thermo-Calc & DICTRA, computational tools for materials science. *Calphad*, 26:273–312, 2002.

[46] L. J. Gibson and M. F. Ashby. *Cellular Solids: Structure and Properties*. Cambridge University Press, 2 edition, 1997.

[47] O. Al-Ketan and R. K. Abu Al-Rub. Multifunctional mechanical metamaterials based on triply periodic minimal surface lattices. *Advanced Engineering Materials*, 21:1900524, 2019.

[48] G. Dong, Y. Tang, and Y. F. Zhao. A 149 line homogenization code for three-dimensional cellular materials written in MATLAB. *Journal of Engineering Materials and Technology*, 141 (1):011005, 2019.

[49] D. R. Jones, M. Schonlau, and W. J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13:455–492, 1998.

[50] M. F. Ashby. *Materials Selection in Mechanical Design*. Butterworth-Heinemann, 4 edition, 2011.

[51] E. George, C. Keefe, P. Pak, and A. Barati Farimani. AgentsCAD: Automated design for manufacturing of FDM parts via multi-agent LLM reasoning and geometric feature recognition. *arXiv preprint arXiv:2607.02448*, 2026.

[52] S. K. Everton, M. Hirsch, P. Stravroulakis, R. K. Leach, and A. T. Clare. Review of in-situ process monitoring and in-situ metrology for metal additive manufacturing. *Materials & Design*, 95:431–445, 2016.

[53] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[54] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lučić, and C. Schmid. ViViT: A video vision transformer. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6836–6846, 2021.